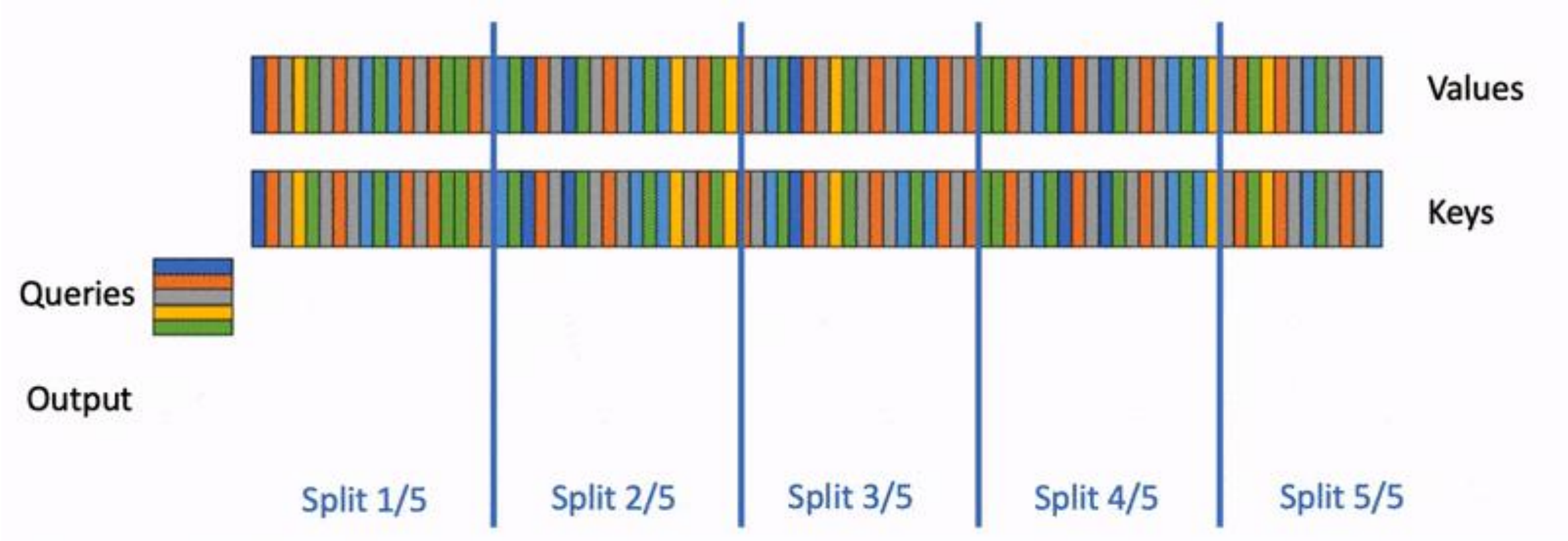
QLoRA

Danyang Zhuo

Duke University

Some materials are from QLora's NeurIPS Slides

FlashDecoding Kernel



What Fine-Tuning?

- Instruction following
  - Sentence completion -> Chatbot
- Domain adaptation
  - General AI -> Mathematician, Lawyer, Doctor
- Private dataset
- Increasing reasoning capability
  - Reinforcement learning

Large models are not easily accessible

Model	Fine-tuning memory
T5-11B	132 GB
Mistral-7B	84 GB
LLaMA2-70B	840 GB

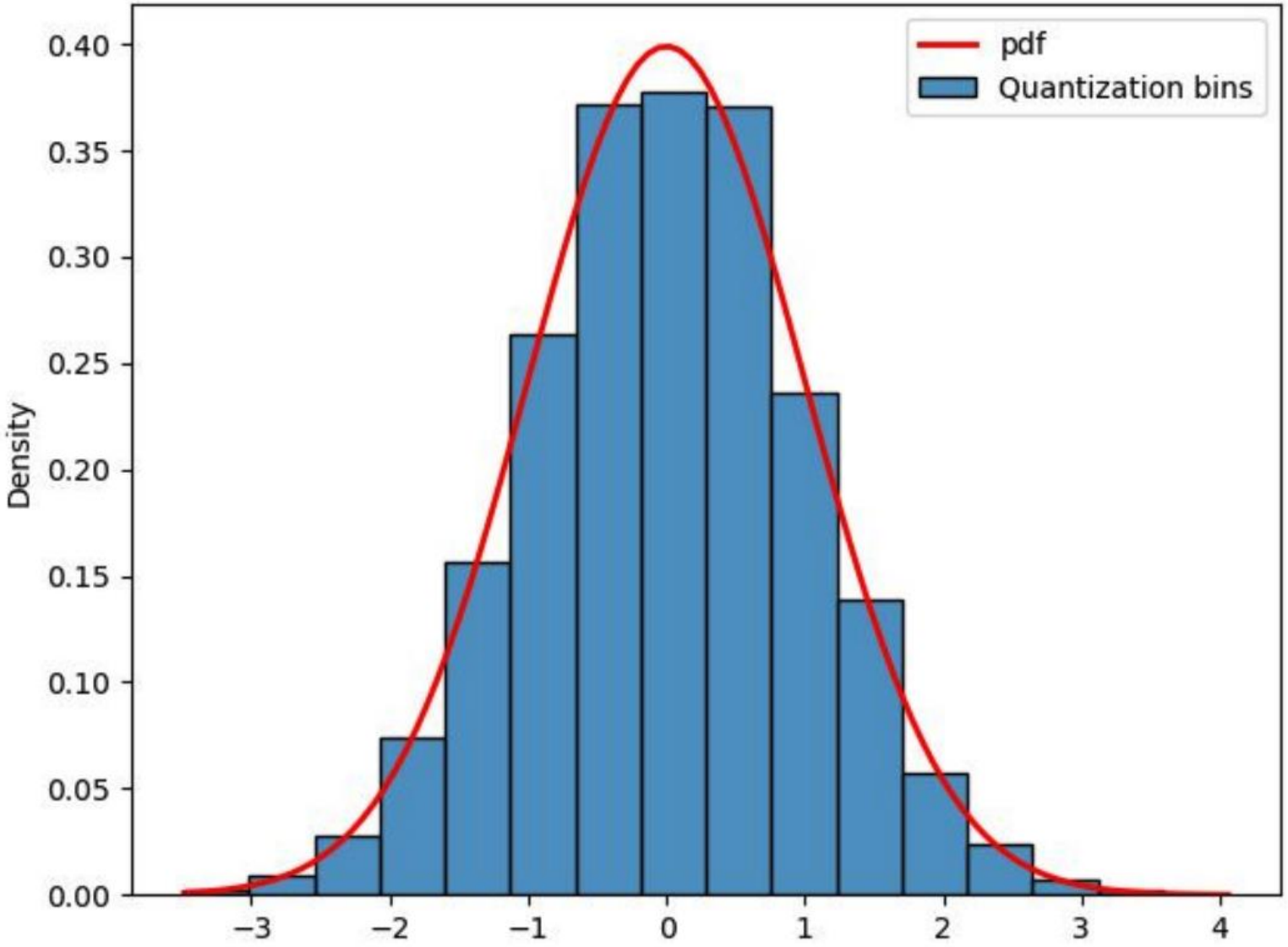
Large models are not easily accessible

Model	Fine-tuning memory
T5-11B	132 GB
Mistral-7B	84 GB
LLaMA2-70B	840 GB

QLoRA

Model	Fine-tuning memory
T5-11B	6 GB
Mistral-7B	5 GB
LLaMA2-70B	46 GB

How does quantization work?



Block-wise k-bit Quantization Quantization is the process of discretizing an input from a representation that holds more information to a representation with less information. It often means taking a data type with more bits and converting it to fewer bits, for example from 32-bit floats to 8-bit Integers. To ensure that the entire range of the low-bit data type is used, the input data type is commonly rescaled into the target data type range through normalization by the absolute maximum of the input elements, which are usually structured as a tensor. For example, quantizing a 32-bit Floating Point (FP32) tensor into a Int8 tensor with range $[-127, 127]$:

$$\mathbf{X}^{\text{Int8}} = \text{round}\left(\frac{127}{\text{absmax}(\mathbf{X}^{\text{FP32}})}\mathbf{X}^{\text{FP32}}\right) = \text{round}(c^{\text{FP32}} \cdot \mathbf{X}^{\text{FP32}}), \tag{1}$$

where c is the *quantization constant* or *quantization scale*. Dequantization is the inverse:

$$\text{dequant}(c^{\text{FP32}}, \mathbf{X}^{\text{Int8}}) = \frac{\mathbf{X}^{\text{Int8}}}{c^{\text{FP32}}} = \mathbf{X}^{\text{FP32}} \tag{2}$$

Quantization as a mapping

Most general form of describe quantization is through a mapping from integers to float values normalized to the range -1.0 and 1.0.

Int4	0	1	2	3	4 …					FP4	0	1	2	3	4 …			
	-7	-6	-5	-4	-3 …						-12	-8	-6	-4	-3 …			
			↓										↓					
	-1	-0.86	-0.71	-0.57	-0.43 …						-1	-0.67	-0.5	-0.33	-0.25 …			

The mapping format { index : float value} generalizes to all data types.

Recipe: How to quantize a tensor?

Given a tensor X of any real data type. We can apply quantization as follows:

1. Normalize X into the range [-1.0, 1.0] by dividing by absmax(X)
2. Find the closest value in the data type (rounding for integers; in general binary search)

Quantization Example: A non-standard 2-bit data type

Map: {Index: 0, 1, 2, 3 -> Values: -1.0, 0.3, 0.5, 1.0}

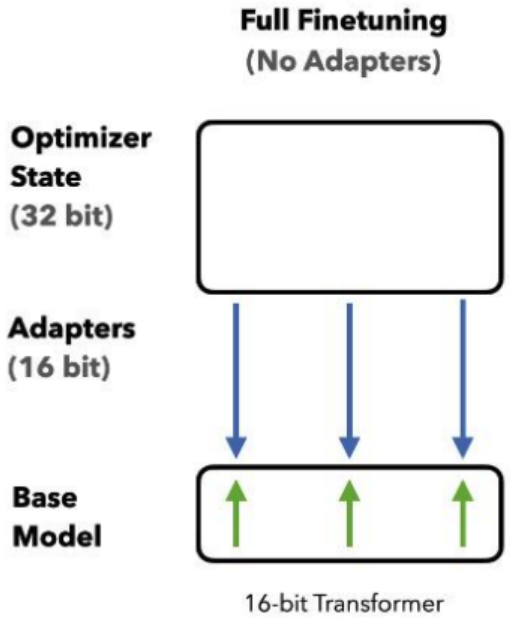
Input tensor: [10, -3, 5, 4]

1. Normalize with absmax: [10, -3, 5, 4] -> [1, -0.3, 0.5, 0.4]
2. Find closest value: [1, -0.3, 0.5, 0.4] -> [1.0, 0.3, 0.5, 0.5]
3. Find the associated index: [1.0, 0.3, 0.5, 0.5] -> [3, 1, 2, 2] -> store
4. Dequantization: load -> [3, 1, 2, 2] -> lookup -> [1.0, 0.3, 0.5, 0.5] -> denormalize -> [10, 3, 5, 5]

Finetuning is expensive.

Finetuning cost per parameter:

- Weight: 16 bit
- Weight gradient: 16 bit
- Optimizer state: 64 bit
- 12 byte per parameter



70B model -> 840 GB of GPU memory -> 36x consumer GPUs

Low-rank Adapters Low-rank Adapter (LoRA) finetuning [28] is a method that reduces memory requirements by using a small set of trainable parameters, often termed adapters, while not updating the full model parameters which remain fixed. Gradients during stochastic gradient descent are passed through the fixed pretrained model weights to the adapter, which is updated to optimize the loss function. LoRA augments a linear projection through an additional factorized projection. Given a projection $\mathbf{XW} = \mathbf{Y}$ with $\mathbf{X} \in \mathbb{R}^{b \times h}$, $\mathbf{W} \in \mathbb{R}^{h \times o}$ LoRA computes:

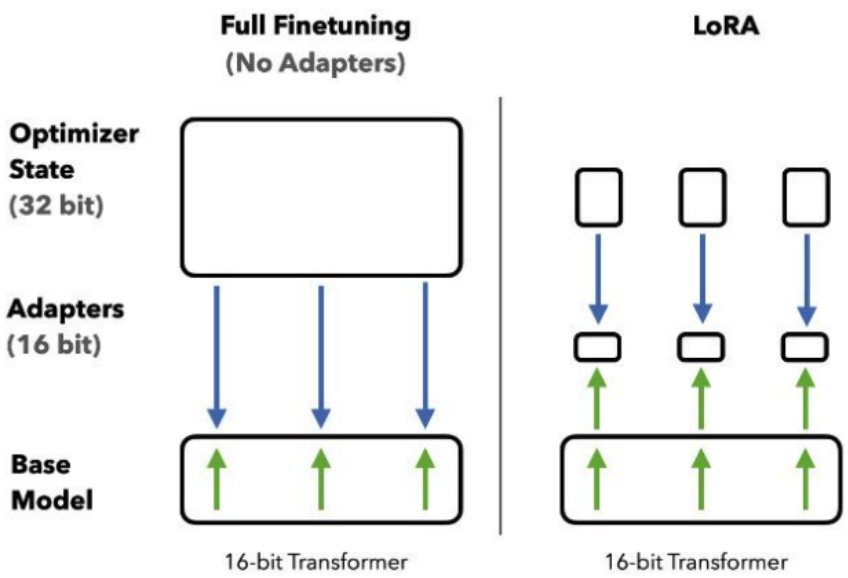
$$\mathbf{Y} = \mathbf{XW} + s\mathbf{XL}_1\mathbf{L}_2, \tag{3}$$

where $\mathbf{L}_1 \in \mathbb{R}^{h \times r}$ and $\mathbf{L}_2 \in \mathbb{R}^{r \times o}$, and s is a scalar.

Finetuning with Low Rank Adapters (LoRA).

Finetuning cost per parameter:

- Weight: 16 bits
- Weight gradient: ~0.4bit
- Optimizer state: ~0.8bit
- Adapter weights: ~0.4bit
- 17.6 bits per parameter



70B model -> 154 GB of GPU memory -> 8x consumer GPUs

QLoRA: 4-bit frozen base model + Low rank Adapters

Finetuning cost per parameter:

- Weight: 4 bit
- Weight gradient: ~0.4 bit
- Optimizer state: ~0.8 bit
- Adapter weights: ~0.4 bit
- 5.2 bit per parameter

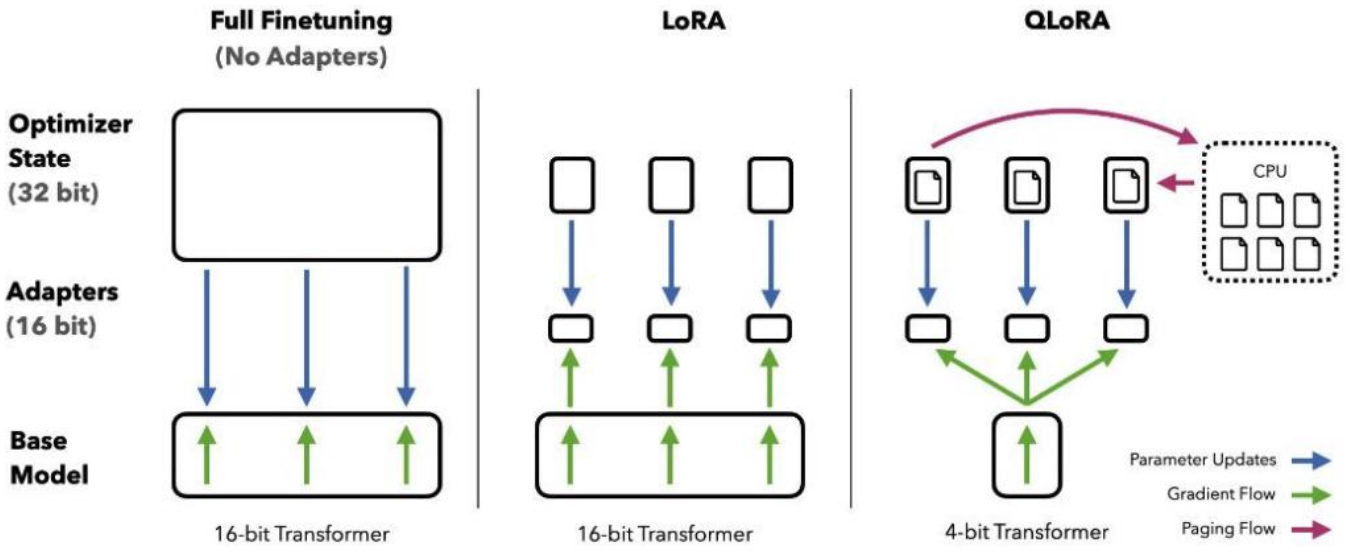
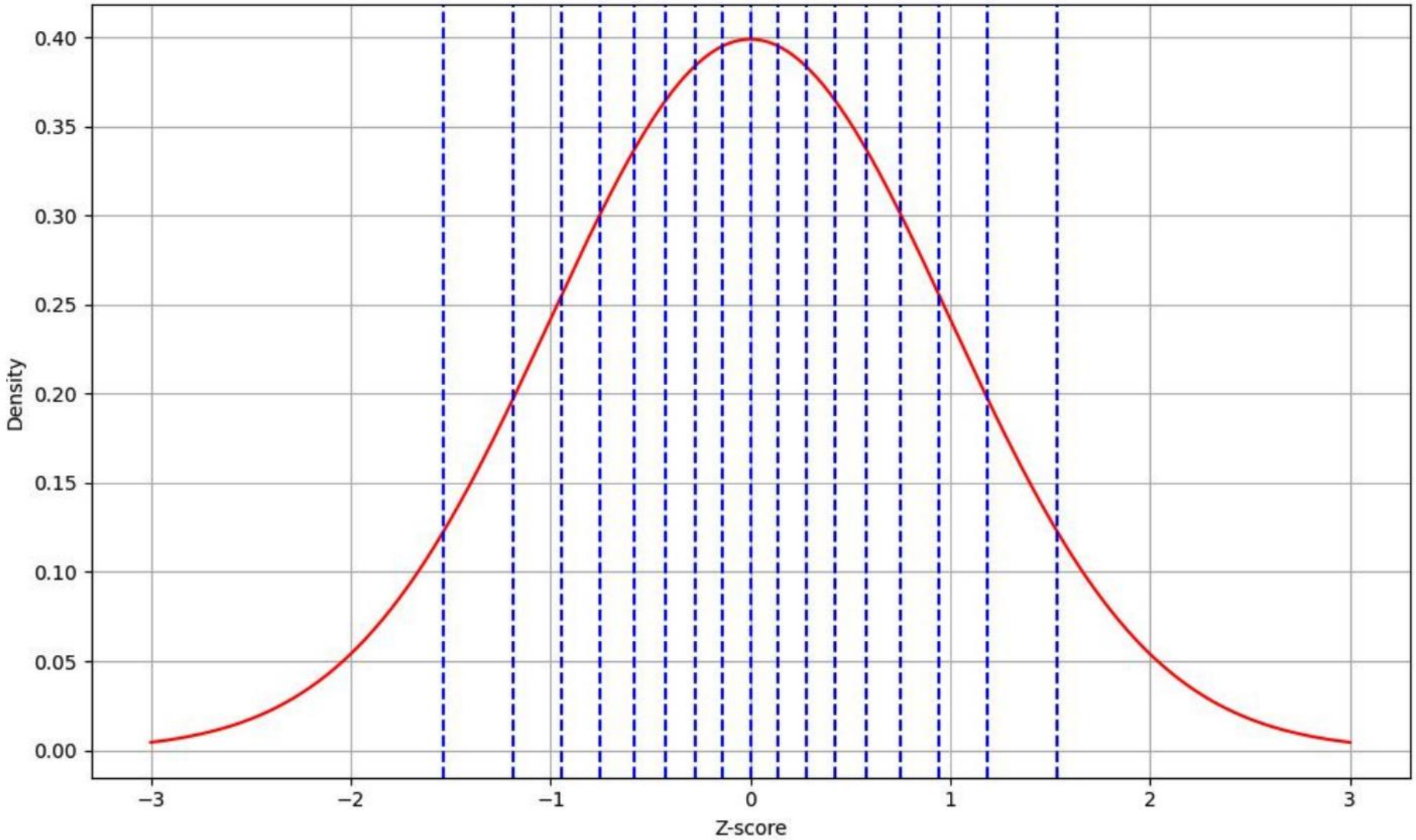


Figure 1: Different finetuning methods and their memory requirements. QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers to handle memory spikes.

70B model -> 46 GB of GPU memory -> 2x consumer GPUs.

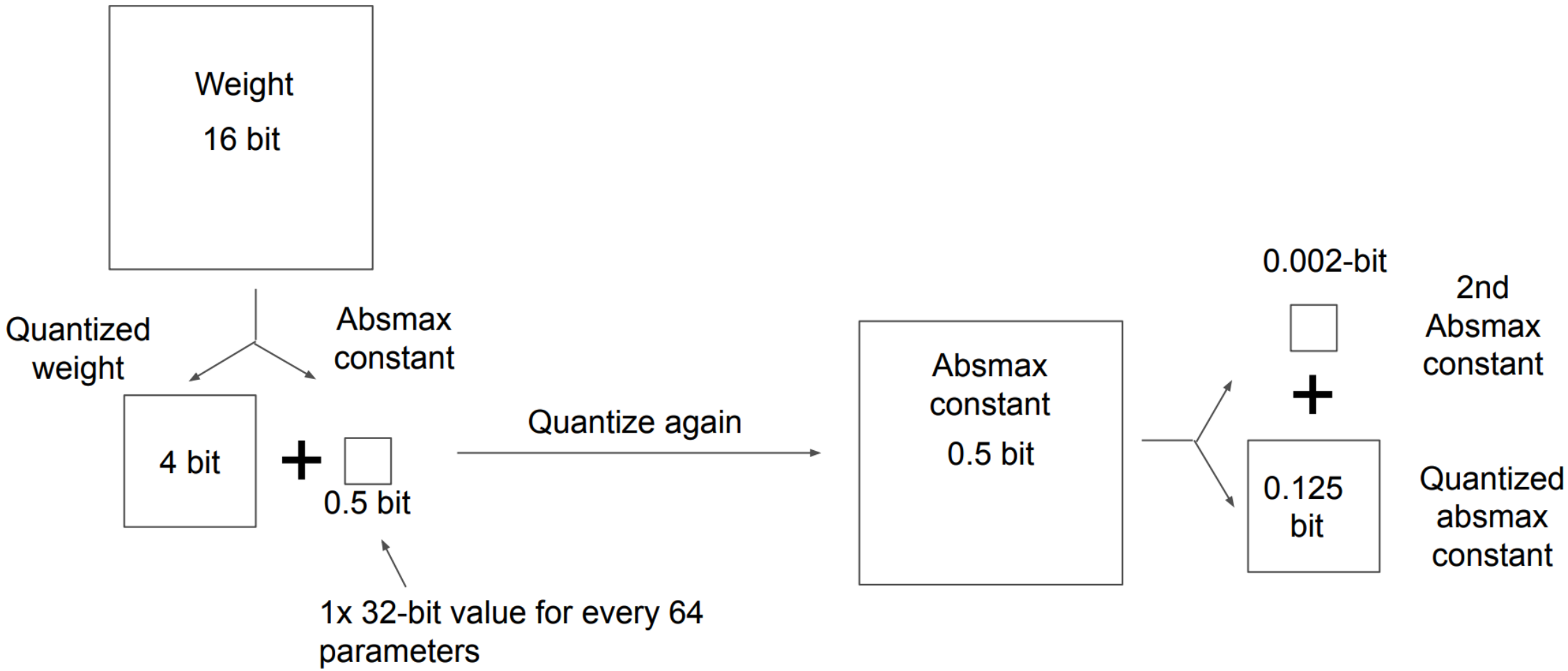
4-bit NormalFloat (NF4) an information-theoretically optimal data type for normal distributions



Why not quantile quantization?

The main limitation of quantile quantization is that the process of quantile estimation is expensive. Therefore fast quantile approximation algorithms, such as SRAM quantiles [15], are used to estimate them. Due to the approximate nature of these quantile estimation algorithms, the data type has large quantization errors for outliers, which are often the most important values.

Reduce absmax constant size with Double Quantization



Paged Optimizers use the NVIDIA unified memory [3] feature wich does automatic page-to-page transfers between the CPU and GPU for error-free GPU processing in the scenario where the GPU occasionally runs out-of-memory. The feature works like regular memory paging between CPU RAM and the disk. We use this feature to allocate paged memory for the optimizer states which are then automatically evicted to CPU RAM when the GPU runs out-of-memory and paged back into GPU memory when the memory is needed in the optimizer update step.

QLoRA. Using the components described above, we define QLoRA for a single linear layer in the quantized base model with a single LoRA adapter as follows:

$$\mathbf{Y}^{\text{BF16}} = \mathbf{X}^{\text{BF16}}\text{doubleDequant}(c_1^{\text{FP32}}, c_2^{\text{k-bit}}, \mathbf{W}^{\text{NF4}}) + \mathbf{X}^{\text{BF16}}\mathbf{L}_1^{\text{BF16}}\mathbf{L}_2^{\text{BF16}}, \tag{5}$$

where doubleDequant$(\cdot)$ is defined as:

$$\text{doubleDequant}(c_1^{\text{FP32}}, c_2^{\text{k-bit}}, \mathbf{W}^{\text{k-bit}}) = \text{dequant}(\text{dequant}(c_1^{\text{FP32}}, c_2^{\text{k-bit}}), \mathbf{W}^{\text{4bit}}) = \mathbf{W}^{\text{BF16}}, \tag{6}$$

Table 7: Elo rating for a tournament between models where models compete to generate the best response for a prompt, judged by human raters or GPT-4. Overall, Guanaco 65B and 33B tend to be preferred to ChatGPT-3.5 on the benchmarks studied. According to human raters they have a Each 10-point difference in Elo is approximately a difference of 1.5% in win-rate.

Benchmark	Vicuna		Vicuna		Open Assistant		
# Prompts	80		80		953		
Judge	Human raters		GPT-4		GPT-4		Median Rank
Model	Elo	Rank	Elo	Rank	Elo	Rank	
GPT-4	1176	1	1348	1	1294	1	1
Guanaco-65B	1023	2	1022	2	1008	3	2
Guanaco-33B	1009	4	992	3	1002	4	4
ChatGPT-3.5 Turbo	916	7	966	5	1015	2	5
Vicuna-13B	984	5	974	4	936	5	5
Guanaco-13B	975	6	913	6	885	6	6
Guanaco-7B	1010	3	879	8	860	7	7
Bard	909	8	902	7	-	-	8

Multi-model inference

- Let’s say a model engineer #1 fine-tunes W to W1
  - Imagine there are 100 such engineers
- How to inference W1, W2, ... , W100?
  - Treat them as independent models.
    - Need many GPUs!
    - Resource under utilization when model is not active or batch size is small

LoRA

- We want to store single copy of W, for fine-tuned models we only store L1, L2 for each fine-tuned model
  - Sizeof(L1 + L2) is around 0.2% of Sizeof(W)

Low-rank Adapters Low-rank Adapter (LoRA) finetuning [28] is a method that reduces memory requirements by using a small set of trainable parameters, often termed adapters, while not updating the full model parameters which remain fixed. Gradients during stochastic gradient descent are passed through the fixed pretrained model weights to the adapter, which is updated to optimize the loss function. LoRA augments a linear projection through an additional factorized projection. Given a projection $\mathbf{XW} = \mathbf{Y}$ with $\mathbf{X} \in \mathbb{R}^{b \times h}$, $\mathbf{W} \in \mathbb{R}^{h \times o}$ LoRA computes:

$$\mathbf{Y} = \mathbf{XW} + s\mathbf{XL}_1\mathbf{L}_2, \tag{3}$$

where $\mathbf{L}_1 \in \mathbb{R}^{h \times r}$ and $\mathbf{L}_2 \in \mathbb{R}^{r \times o}$, and s is a scalar.

PUNICA: MULTI-TENANT LORA SERVING

Lequn Chen [*1] **Zihao Ye** [*1] **Yongji Wu** [2] **Danyang Zhuo** [2] **Luis Ceze** [1 3] **Arvind Krishnamurthy** [1]

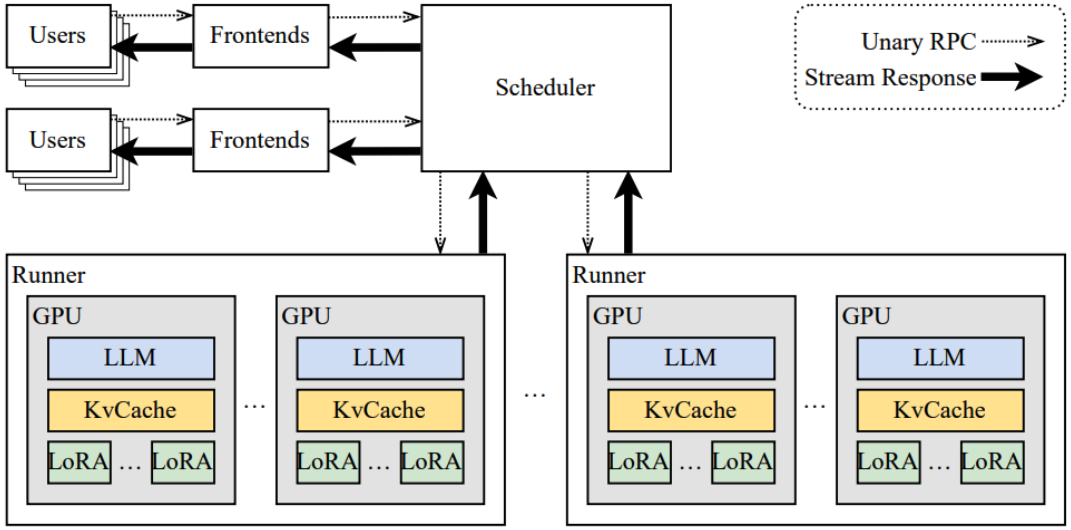


Figure 2. The system architecture of Punica.

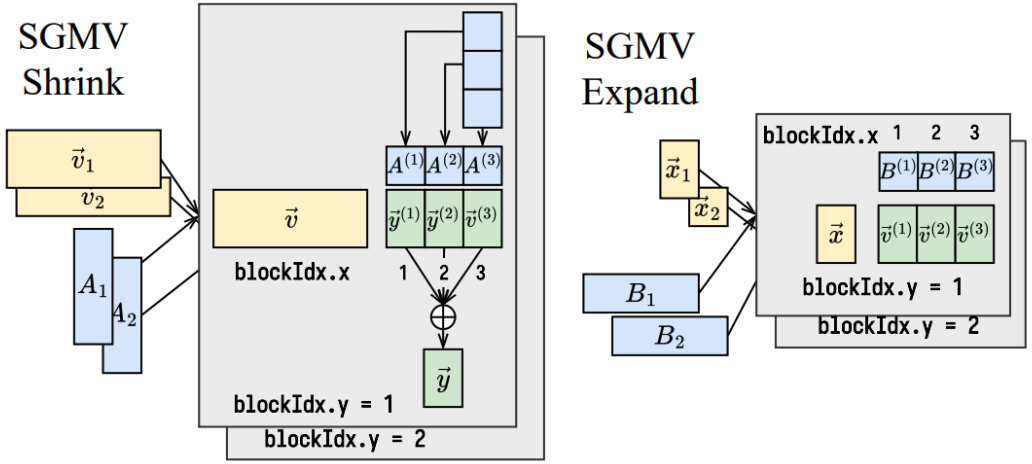


Figure 4. Scheduling of SGMV expand/shrink kernels

Scaling Laws for Neural Language Models

Jared Kaplan *
Johns Hopkins University, OpenAI
jaredk@jhu.edu

Sam McCandlish*
OpenAI
sam@openai.com

Tom Henighan
OpenAI
henighan@openai.com

Tom B. Brown
OpenAI
tom@openai.com

Benjamin Chess
OpenAI
bchess@openai.com

Rewon Child
OpenAI
rewon@openai.com

Scott Gray
OpenAI
scott@openai.com

Alec Radford
OpenAI
alec@openai.com

Jeffrey Wu
OpenAI
jeffwu@openai.com

Dario Amodei
OpenAI
damodei@openai.com

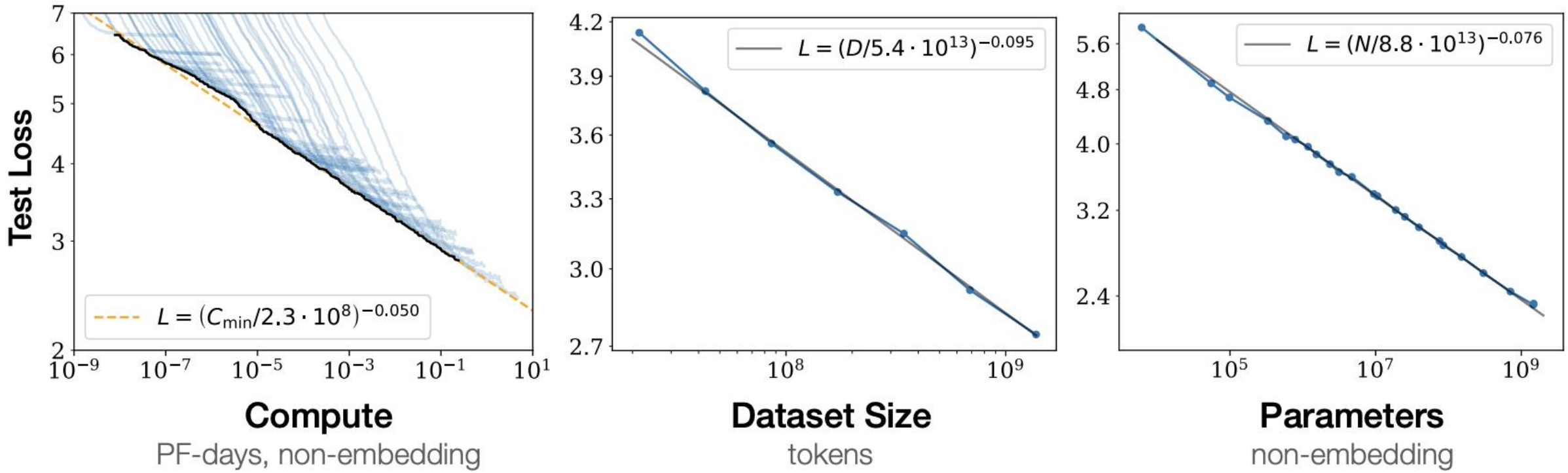


Figure 1 Language modeling performance improves smoothly as we increase the model size, datasetset size, and amount of compute[2] used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

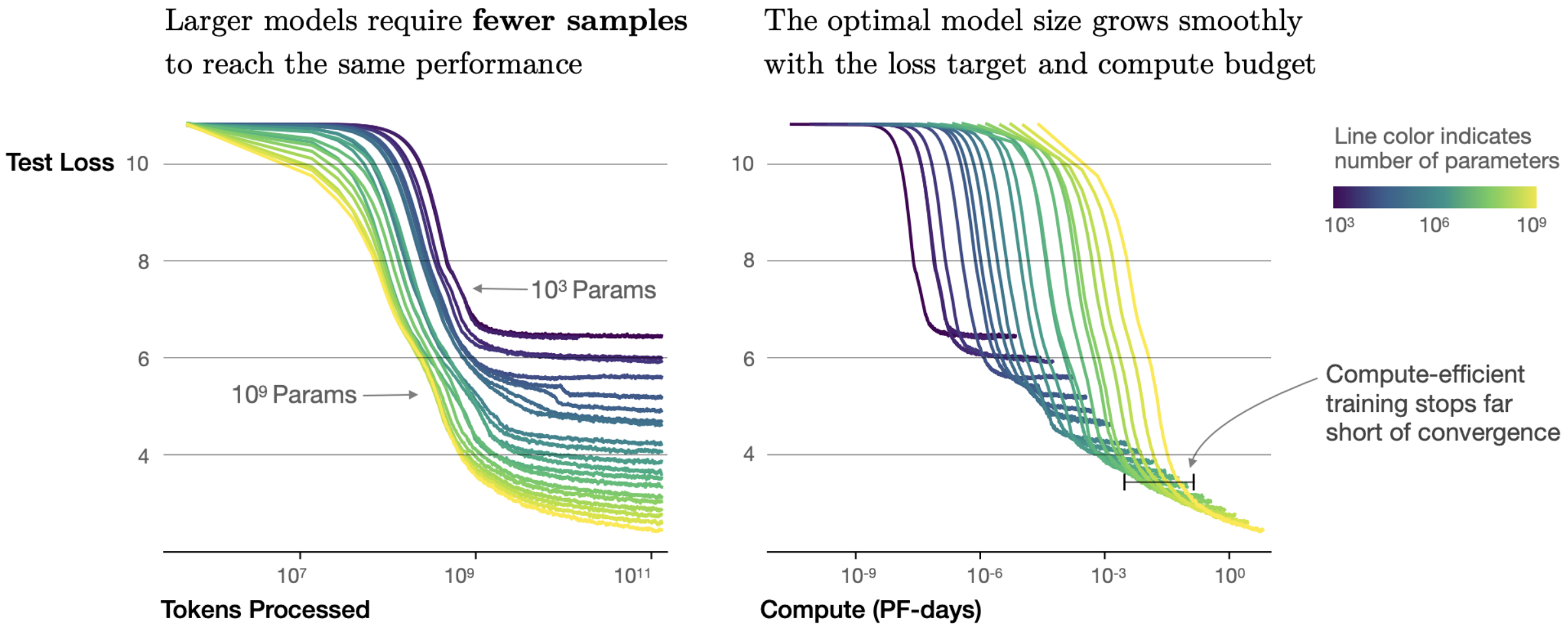


Figure 2 We show a series of language model training runs, with models ranging in size from 10^3 to 10^9 parameters (excluding embeddings).

Given a compute budget

- How large of a model we can train?
- How much training data we need to collect?

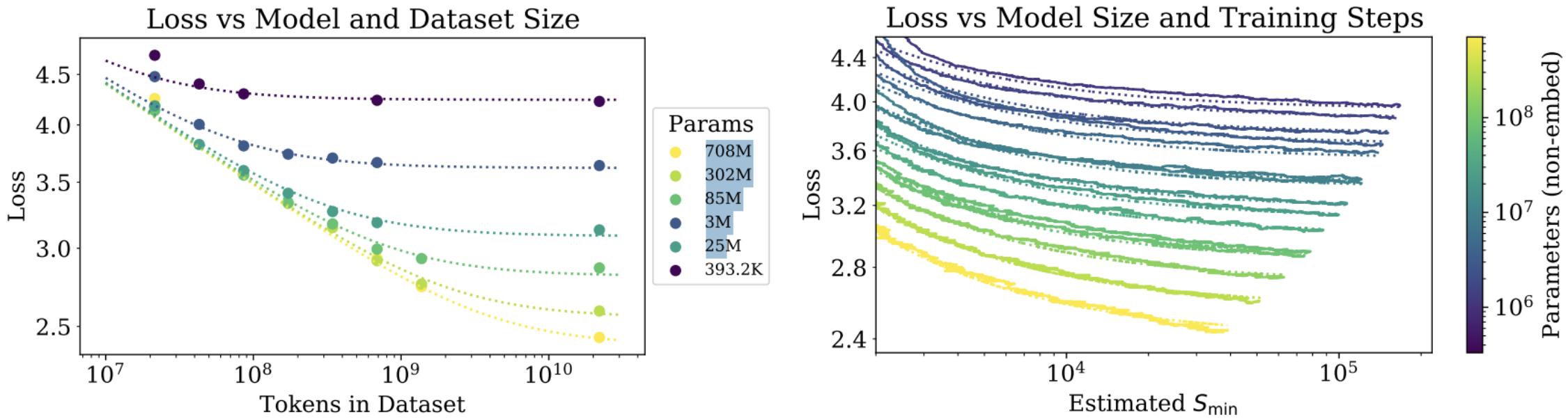


Figure 4 **Left**: The early-stopped test loss $L(N, D)$ varies predictably with the dataset size D and model size N according to Equation (1.5). **Right**: After an initial transient period, learning curves for all model sizes N can be fit with Equation (1.6), which is parameterized in terms of $S_{\min}$, the number of steps when training at large batch size (details in Section 5.1).

$$L(N, D) = \left[\left(\frac{N_c}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D}$$

- L – the cross entropy loss in nats. Typically it will be averaged over the tokens in a context, but in some cases we report the loss for specific tokens within the context.
- N – the number of model parameters, *excluding all vocabulary and positional embeddings*
- $C \approx 6NBS$ – an estimate of the total non-embedding training compute, where B is the batch size, and S is the number of training steps (ie parameter updates). We quote numerical values in PF-days, where one PF-day $= 10^{15} \times 24 \times 3600 = 8.64 \times 10^{19}$ floating point operations.
- D – the dataset size in tokens

$$\begin{aligned} N &\approx 2d_{\text{model}} n_{\text{layer}} \left(2d_{\text{attn}} + d_{\text{ff}}\right) \\ &= 12 n_{\text{layer}} d_{\text{model}}^2 \quad \text{with the standard} \quad d_{\text{attn}} = d_{\text{ff}}/4 = d_{\text{model}} \end{aligned}$$

Operation	Parameters	FLOPs per Token
Embed	$(n_{\text{vocab}} + n_{\text{ctx}})\, d_{\text{model}}$	$4d_{\text{model}}$
Attention: QKV	$n_{\text{layer}} d_{\text{model}} 3d_{\text{attn}}$	$2n_{\text{layer}} d_{\text{model}} 3d_{\text{attn}}$
Attention: Mask	—	$2n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}}$
Attention: Project	$n_{\text{layer}} d_{\text{attn}} d_{\text{model}}$	$2n_{\text{layer}} d_{\text{attn}} d_{\text{embd}}$
Feedforward	$n_{\text{layer}} 2d_{\text{model}} d_{\text{ff}}$	$2n_{\text{layer}} 2d_{\text{model}} d_{\text{ff}}$
De-embed	—	$2d_{\text{model}} n_{\text{vocab}}$
Total (Non-Embedding)	$N = 2d_{\text{model}} n_{\text{layer}} (2d_{\text{attn}} + d_{\text{ff}})$	$C_{\text{forward}} = 2N + 2n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}}$

Table 1 Parameter counts and compute (forward pass) estimates for a Transformer model. Sub-leading terms such as nonlinearities, biases, and layer normalization are omitted.